Programación y Técnicas Computacionales • Python • Unidad 1

Boletín de ejercicios extra

Fundamentos de Python

David Ferro Costas • Theoretical Chemistry and Computational Modelling • 2026/27

Ejercicios adicionales para practicar los contenidos de la Unidad 1.

Ejercicio 1 Un punto dentro de una circunferencia

Imagina una circunferencia de radio R centrada en el origen de coordenadas $(0, 0)$. Escribe un programa que pida el radio de dicha circunferencia y también las coordenadas x e y de un punto cualquiera. El programa debe indicar si dicho punto está contenido (o no) dentro de la circunferencia.

Ejercicio 2 Hebras complementarias de ADN

Escribe un programa que pida dos hebras de ADN, `hebra1` y `hebra2`. El programa debe indicar si `hebra2` es la hebra complementaria de `hebra1`. Si `hebra2` no es la hebra complementaria, el programa debe indicar las posiciones en las que las bases no se complementan. El programa también debe imprimir el grado de diferencia (en tanto por ciento) entre `hebra2` y la verdadera complementaria de `hebra1`.

> **Recuerda.** En la cadena complementaria del ADN, `A` se sustituye por `T`, `T` por `A`, `G` por `C` y `C` por `G`. Así, la hebra complementaria de `ATTGCGA` es `TAACGCT`.

> **Restricción.** En este ejercicio no se puede usar la función `len()`.

Pasos sugeridos

(1) Haz que el programa pida dos hebras usando dos veces la función `input`. Para facilitar la explicación, imaginemos que el usuario introduce las hebras `ATTGCGA` y `TACCGAT`.

(2) Calcula la longitud de una de las hebras (vamos a suponer que las dos tienen la misma longitud). Para ello, puedes usar un bucle `for-in` con un contador que valga 0 fuera del bucle y que, por cada carácter de la hebra, aumente su valor en una unidad. En el caso del ejemplo, debes obtener un valor de 7.

(3) Compara las dos hebras con otro bucle `for-in` (usa `range` y la longitud calculada en el punto 2). El bucle te debe permitir comparar el carácter i-ésimo de la primera hebra con el carácter i-ésimo de la segunda hebra (siendo i el número que genera el bucle `for-in` en cada iteración). Por ejemplo, la posición 0 corresponde, en el ejemplo, a `A` y `T`, que son complementarias. En la posición 2 tenemos `T` y `C`, que no son complementarias. En este último caso, debes aumentar en 1 el valor de un contador externo que lleve la cuenta del número de diferencias. El programa también debe imprimir que en la posición 2 las hebras no son complementarias.

(4) Finalmente, sabiendo en cuántas posiciones difieren (punto 3) y la longitud de las hebras (punto 2), calcula el porcentaje de diferencia entre la segunda hebra y la complementaria real de la primera hebra.

Ejercicio 3 Codones de una secuencia de ARN

Escribe un programa que pida una secuencia de ARN y encuentre el codón de iniciación `AUG`. Una vez encontrado, el programa debe leer el ARN en codones hasta encontrar el codón de finalización `UGA`. El programa debe imprimir los codones encontrados entre el codón de iniciación y el de finalización.

Restricción. En este ejercicio no se puede usar la función `len()`.

Pasos sugeridos

(1) Haz que el programa pida una secuencia de ARN usando la función `input`. Imaginemos que el usuario introduce la secuencia:

CACCAU**AUG**UUCAUAGACGAGUUAAUCAGC**UGA**AAA

(2) Calcula la longitud de la secuencia. Para ello, puedes usar un bucle `for-in` sobre la secuencia, con un contador que valga 0 fuera del bucle y que, por cada carácter de la secuencia, aumente su valor en una unidad. En el caso del ejemplo, debes obtener un valor de 36.

(3) Usa un bucle `for-in` combinado con `range` y la longitud calculada en el punto 2. Dado que los codones van de 3 en 3, el bucle debe generar números separados de 3 en 3 (es decir: 0, 3, 6, 9, 12, 15...). Para cada número i del bucle, el codón va desde i hasta $i+3$, por lo que debes usar *slicing* para obtener los tres caracteres del codón.

En el momento en que la secuencia de 3 caracteres sea `AUG`, el programa debe empezar a imprimir los codones visitados. Para ello, te recomiendo crear una variable igual a `False` fuera del bucle que se convierta en `True` cuando aparezca `AUG`. Del mismo modo, cuando aparezca `UGA`, esta variable debería pasar a valer `False`. Cuando esta variable valga `True`, el programa debe imprimir, dentro del bucle, el codón. En el ejemplo, el programa debe imprimir los siguientes codones:

(a) `UUC`
(b) `AUA`
(c) `GAC`
(d) `GAG`
(e) `UUA`
(f) `AUC`
(g) `AGC`

Ejercicio 4 El alimento más y menos calórico

Considera el siguiente diccionario, que contiene el número de kilocalorías (por cada 100 gramos) de varios alimentos:

```
alimentos = {"cebolla": 40, "patatas": 86, "moras": 43,
             "fresas": 32, "guisantes": 82}
```

Haz un programa que, usando un bucle `for-in`, sea capaz de indicar qué alimento presenta el menor valor calórico y cuál el mayor.

Ejercicio 5 Un mensaje secreto

Un amigo te ha enviado el siguiente mensaje secreto:

DOSMIEZDFEISADSMGHBEYSDFSXYSCPTDNCO

Para descifrarlo, escribe un programa en Python que use el siguiente diccionario:

```
secreto = {"Q": "A", "T": "B", "N": "C", "A": "D",
           "D": "E", "Z": "F", "K": "G", "B": "H",
           "C": "I", "W": "J", "S": " ", "O": "L",
           "P": "M", "Y": "N", "E": "O", "M": "P",
           "R": "Q", "I": "R", "F": "S", "H": "T",
           "X": "U", "L": "V", "J": "W", "V": "X",
           "G": "Y", "U": "Z", " ": "K"}
```

Este diccionario te indica cómo convertir cada letra del código para leer el mensaje correctamente.

Recuerda que las cadenas de texto se pueden concatenar, de forma que puedes usar un bucle para ir añadiendo letras a una variable que contenga una cadena. Así, te recomiendo que crees la siguiente variable:

```
mensaje_traducido = ""
```

y que, usando un bucle `for-in` y el diccionario, le vayas añadiendo cada letra descifrada.